Android Interview — Night Before

All 11 cheat sheets distilled · ~20 minutes to read · then sleep

~6 pages | Pure signal | Read last | Don't learn — recall

How to use this page: read top to bottom in one sitting (~20-30 min). Then close it, drink water, sleep 7+ hours. Don't try to learn new things — this is for last-mile recall of what you already know. If a section feels unfamiliar, you have NOT learned it; ignore the temptation to cram.

1. The 10 universal answers 5 min

These come up in *every* Android interview regardless of the system you're discussing. Memorize the one-liner.

Q: How do you handle process death?

SavedStateHandle for in-flight UI state · Room/DataStore for durable data · WorkManager for queued work · on relaunch, restore from local before any network call.

Q: How do you handle offline?

Room is the source of truth · UI observes a Flow from Room · writes go to Room first then an outbox · sync engine drains outbox via WorkManager when network returns.

Q: How do you handle authentication?

OkHttp Interceptor adds the access token to every request · OkHttp Authenticator reactively refreshes on 401 and retries · refresh token in EncryptedSharedPreferences (Keystore-backed).

Q: How do you minimize battery impact?

WorkManager with NetworkType.UNMETERED + BatteryNotLow constraints · batch network calls · avoid wake locks · respect Doze.

Q: How do you handle config changes (rotation)?

ViewModel survives · `stateIn(WhileSubscribed(5000))` keeps upstream alive across the brief detach · SavedStateHandle for primitive UI state.

Q: How do you test this?

Repository as the test seam · fake data sources implement the same interface · ViewModel tested via runTest + Turbine · UI via Compose-test with fake repositories.

Q: How do you instrument observability?

Structured analytics events · traceparent header for distributed tracing · Crashlytics for stability · Firebase Performance for cold start & latency.

Q: How does this scale to 100M users?

Mobile-side: this is constant per device · backend: stateless API tier, partition by user_id, CDN for static assets, push-based fanout has issues with celebrities, fall back to pull or hybrid.

Q: Encryption at rest?

Sensitive blobs via EncryptedSharedPreferences / EncryptedFile · Room SQLite via SQLCipher if needed · secrets in Android Keystore (hardware-backed where TEE available).

Q: What about iOS parity?

Architecture is platform-agnostic above the local-storage layer · share API contracts via OpenAPI · share business rules via KMP if the team adopts it · don't share UI.

2. The 90-second universal opening (memorize verbatim) 3 min

Sec 0-20 — Frame requirements: "Before I dive in, let me confirm scope. The functional asks are *X, Y, Z*. Non-functional dimensions I'll explicitly call out: offline-first, battery, p95 latency. Any constraints on regions or scale I should know?"

Sec 20-40 — Estimate scale: "Roughly N DAU × M sessions × K events ≈ X RPS peak and Y MB per user of local storage. Fits comfortably in Room; means a single OkHttp client with HTTP/2 connection pool."

Sec 40-60 — Architecture: "Single source of truth in Room with unidirectional flow: Repository → ViewModel (StateFlow) → Compose UI. Network responses write into Room; UI only reads from Room. Gives offline support, predictable rotation/process death behavior, clean test seam."

Sec 60-80 — Components: "OkHttp + Retrofit with auth interceptor and refresh Authenticator; real-time via WebSocket wrapped in callbackFlow with exponential backoff; background work via WorkManager with NetworkType.CONNECTED + idempotency keys."

Sec 80-90 — Pre-empt failures: "Failure modes I want to address: token expiry mid-request, WebSocket drops, process death mid-write, Room migration failures. I'll cover each as I walk through."

Say only that, and you've separated yourself from 70% of candidates. Now you can actually start designing.

3. Architecture — pick one in 10 seconds 2 min

USE	PICK
New Compose app, simple-to-medium screens	MVVM
Complex async screens (chat, video, multi-step forms)	MVI for those screens; MVVM for the rest
Multi-platform (Android + iOS via KMP)	Clean Architecture in shared module + MVVM/MVI per platform UI
Legacy Java codebase	MVP for incremental migration
Multi-team enterprise	Multi-module + Clean + MVVM/MVI per feature

Where things go (universal):

- Network/DB calls → inside Repository; never in ViewModel
- WorkManager → enqueued from Repository or UseCase

- ForegroundService → started from Application or controller VM
- ContentProvider → wrap in DataSource interface
- BroadcastReceiver → notify Repository → it updates state Flow
- Hilt → @HiltAndroidApp on App, @HiltViewModel on VMs, @Module for repos
- One-shot events → Channel or SharedFlow (replay=0); never StateFlow

4. Coroutines & Flow 2 min

Mental model: compiler converts each suspend into a state machine taking a Continuation. Suspension means storing state and returning `COROUTINE_SUSPENDED`; resumption is a callback that jumps to the next state. No threads held while waiting.

Builders: `launch` = fire-and-forget · `async` = need result (parallel) · `runBlocking` = tests/bridging only · `coroutineScope { }` = all-or-nothing parallel · `supervisorScope { }` = independent parallel · `withContext(d)` = switch dispatcher · `withTimeout(ms)`.

Dispatchers: Main = UI · Default = CPU · IO = blocking I/O · `Default` and `IO` share the SAME thread pool (only the cap differs).

Structured concurrency: Parent waits for children · cancel parent → cancels all children · child fails → cancels parent (unless supervisor) · cancellation = `CancellationException` at suspension points · cooperative — tight CPU loop needs `yield()`.

Hot vs Cold: Cold (`flow { }`) restarts per collector. Hot: `StateFlow` (state + initial), `SharedFlow` (events + replay), `Channel` (point-to-point). For UI: `stateIn(WhileSubscribed(5000))`.

Operators you'll use: `map`, `filter`, `distinctUntilChanged`, `debounce`, `flatMapLatest` (search), `combine` (latest of all), `flowOn`, `buffer`, `conflate`, `onEach`, `catch`, `retry`.

Scope decision: ViewModel → `viewModelScope` · Composable event → `rememberCoroutineScope()` · Activity/Fragment View → `lifecycleScope` + `repeatOnLifecycle(STARTED)` · Worker → `coroutineScope { }` inside doWork · long-lived component → custom `CoroutineScope(SupervisorJob() + dispatcher)` with manual cancel · **NEVER** `GlobalScope`.

Testing: `runTest` + virtual time + `StandardTestDispatcher` · `Dispatchers.setMain(d)` in setUp · `Turbine.test { awaitItem() }`.

5. Jetpack Compose 2 min

Mental model: UI = f(state). Compose Compiler rewrites every `@Composable` to inject a Composer + change-tracking bitmask + restart group. Composer maintains the slot table (flat gap-buffer array). Snapshot system tracks State reads per RecomposeScope. State change → smallest enclosing scope re-runs next frame.

State: `mutableStateOf` / `mutableIntStateOf` · `remember` for in-composition, `rememberSaveable` for process death · `derivedStateOf` + `remember` for noisy inputs → rare output. Hoist state to lowest common ancestor.

Side effects (decision tree): `LaunchedEffect(key)` — coroutine, restart on key change · `DisposableEffect(key)` — register/unregister with onDispose · `SideEffect` — publish to non-Compose code ·

`rememberCoroutineScope()` — launch from event handlers · `produceState` — async source → State · `rememberUpdatedState` — capture latest in long-running effect.

Performance — the 5 fixes:

1. Read Compose Compiler stability report (every Composable should be `skippable`)
2. For unstable params, use `ImmutableList` from kotlinx.collections.immutable or annotate `@Immutable`
3. Defer State reads to the lowest level — read `name` in a leaf, not the whole UiState in a parent
4. For animated values, use `Modifier.graphicsLayer { }` (drawing phase) over value-modifiers like `Modifier.rotate`
5. Stable keys in lazy lists — entity ID, never index

3 phases per frame: Composition → Layout → Drawing. Modifiers with lambdas (`offset { }`, `graphicsLayer { }`) defer reads to later phases — no recomposition.

6. Hilt & DI 1 min

Setup: `@HiltAndroidApp` on Application · `@AndroidEntryPoint` on Activity/Fragment/Service · `@HiltViewModel` on ViewModels · `@Module @InstallIn(...)` for `@Provides` / `@Binds` · KSP plugin for processing.

Bindings: `@Provides` for 3rd-party / custom construction · `@Binds` (abstract) for interface → impl · `@IntoSet` / `@IntoMap` for multibinding · `@Qualifier` for same-type disambiguation · `@AssistedInject + @Assisted` for runtime params (Workers).

Scopes (smallest that fits): `@Singleton` (app) · `@ActivityRetainedScoped` (rotation-survives, multi-Fragment shared state) · `@ViewModelScoped` · `@ActivityScoped` · `@FragmentScoped` · no annotation = fresh per request.

Critical: `@Singleton` holding Activity reference = leak · KMP needs Koin (Hilt is Android-only).

7. Testing 1 min

Stack: JUnit 5 (JVM) / Kotest (KMP) · MockK (mocks) + hand-written fakes (preferred for repos) · kotlinx-coroutines-test + Turbine · MockWebServer for HTTP · Room in-memory for DAO · Robolectric for Android API on JVM · Compose Test rule for UI · Hilt testing for DI integration · Paparazzi/Roborazzi for screenshots.

Layer → approach:

- Plain class / UseCase / ViewModel / Repository → JUnit 5 + fakes
- Room DAO → instrumented + in-memory DB
- Retrofit Api → JUnit 5 + MockWebServer
- Composable → Compose test rule (hoist state for testability)
- Service → extract logic to plain class, test that
- ContentProvider → wrap in DataSource interface, test through that

Testability principles: constructor injection · depend on interfaces · inject dispatcher + clock · pure functions · hoist Compose state · no hidden global state.

Avoid: `runBlocking` in tests · mocks for repositories (use fakes) · testing private methods · shared mutable fixtures · `Thread.sleep` · pursuing 100% coverage.

8. Performance + Security 2 min

Perf — biggest levers:

- Baseline Profile (cold start +20-40%, scroll +30-50%)
- R8 minify + shrink (APK -30-60%)
- App Startup library (consolidate ContentProviders)
- StrictMode in debug; never main-thread I/O in prod
- Compose: stable params, defer reads to lower phases, derivedStateOf
- Coil/Glide HARDWARE bitmap + downsample at decode
- WorkManager constraints (UNMETERED, BatteryNotLow)
- KSP over kapt

Security — must-have layers:

- Network Security Config + cert pinning + at least 1 backup pin
- Keystore-backed encryption for refresh tokens
- BiometricPrompt + CryptoObject for high-stakes ops
- Play Integrity for signup / payment endpoints
- R8 obfuscation + mapping.txt to Crashlytics + Play
- App Links + autoVerify + assetlinks.json
- Custom Tabs over WebView for external URLs
- Photo Picker / Scoped Storage over READ_EXTERNAL_STORAGE
- android:allowBackup=false (or per-file rules)

9. Gradle 1 min

3 phases per build: Initialization → Configuration → Execution. Configuration is expensive; configuration cache skips it.

Build types vs flavors vs variants: Build type = HOW (debug/release) · Flavor = WHAT (qa/prod, free/paid) · Variant = flavor × buildType (qaDebug, prodRelease) · Per-variant configs: `debugImplementation`, `qaImplementation`.

Dependency configs: `implementation` (default; hidden from consumers) · `api` (exposed; consumers see types) · `compileOnly` / `runtimeOnly` · `ksp` / `kapt` · `testImplementation` / `androidTestImplementation`.

Performance trio in `gradle.properties`:

- `org.gradle.daemon=true`
- `org.gradle.parallel=true`
- `org.gradle.caching=true`
- `org.gradle.configuration-cache=true`

Critical clarification: Build scripts (Groovy/Kotlin DSL) are build-time only — they run on developer/CI machine. The APK ships compiled DEX bytecode + resources + manifest + signing. NO Groovy, NO build scripts in the APK.

10. CI/CD 1 min

Definitions: CI = every push automatically built + tested. CD = every successful build automatically ready (Delivery) or goes (Deployment) to users.

Pipeline phases: Trigger → Checkout → Setup JDK + SDK → Restore Gradle cache → Lint → Unit tests → Build AAB → Sign → Upload to Play Store → Notify.

Tools: Personal/hobby = GitHub Actions · mobile-first team = Bitrise / Codemagic · monorepo = CircleCI · enterprise self-hosted = Jenkins / GitLab CI.

Skills needed: YAML (2-3 hours) · Bash basics · Gradle CLI (assembleRelease, bundleRelease) · optional: Fastlane lanes, Jenkins Groovy.

Android-specific: Sign with keystore from base64 secret · use product flavors for environments · upload AAB (not APK) · always upload mapping.txt · use Gradle Play Publisher or Fastlane `upload_to_play_store` · staged rollout for prod (10% → 50% → 100%).

11. Mobile System Design — universal opening + 5 most-asked 2 min

The 90-second universal opening (memorized in §2 above) handles the framing for ANY system you're asked. Beyond the opening, here are the 5 systems most likely to come up:

SYSTEM	THE ONE MENTAL HOOK
Image Loader (Glide-style)	Pipeline: mem → disk → net → decode w/ inSampleSize → cache write. Lifecycle-bound; BitmapPool reuse.
Chat (WhatsApp)	WS in foreground + FCM wakeup; outbox + idempotent UUID; Lamport clock; E2EE via Signal protocol; multi-device fanout per recipient.
Social Feed (Twitter/IG)	Pager + RemoteMediator + cursor; Room is truth; single ExoPlayer to most-visible video; impressions after 50% visible & 500 ms dwell.
Uber Rider	Sealed RideState state machine (Idle → Searching → Matched → OnTrip → Completed); WS for driver dots filtered to viewport; ForegroundService during trip.
Spotify (offline music)	MediaSessionService + ExoPlayer + AudioFocus + becoming-noisy receiver; SimpleCache + Widevine offline license; gapless via ExoPlayer queue.

The 6-step framework that works for ANY new system you get thrown:

1. Functional + non-functional requirements (offline, battery, p95 latency)
2. Back-of-envelope scale (DAU × sessions × events → RPS, MB/user)
3. Architecture (Compose ← StateFlow ← ViewModel ← Repository ← Room/Network)
4. Component walkthrough (OkHttp, Retrofit, WorkManager, etc.)
5. Failure modes (token expiry, WS drops, process death, migration)
6. Trade-offs + alternatives (push vs pull, REST vs WS, etc.)

12. Anti-patterns that drop you a level 1 min

- Treating the phone as a thin client — saying "the client just calls REST" instead of using Room/WorkManager/Keystore
- Calling network from UI/ViewModel directly — always: UI ← Room ← Repository ← Network
- Saying "I'll use MVVM" without explaining state ownership / who mutates / how rotation+process death are handled
- Ignoring battery — no WorkManager constraints mentioned
- Using `runBlocking` or `GlobalScope`
- Floats/doubles for money — always integer cents or BigDecimal
- Skipping idempotency on create/purchase
- "I'll add caching later" — caching is a design decision with TTL + eviction
- Forgetting process death — "what if OS kills my app right here?" is the question that separates Senior from Staff
- Hand-waving conflict resolution — "server timestamp wins; loser preserved as conflict copy" or "CRDT for collaborative"
- No mention of observability (Crashlytics, traces, metrics)

13. If you forget everything else, remember this read twice

The 7 things you MUST be able to say

1. **"Room is the source of truth; UI observes a Flow from Room."** Solves offline + rotation + process death + testing in one sentence.
2. **"OkHttp Interceptor adds the token; Authenticator refreshes on 401."** Solves auth.
3. **"WorkManager with constraints (UNMETERED, BatteryNotLow) and idempotency keys."** Solves background + battery + retries.
4. **"SavedStateHandle for in-flight state; ViewModel survives rotation; Room survives process death."** Solves Android lifecycle.
5. **"`stateIn(WhileSubscribed(5000))` + `collectAsStateWithLifecycle`."** Solves modern Compose state collection.
6. **"Sealed UiState class + StateFlow; one-shot events via SharedFlow(replay=0)."** Solves UI state modeling cleanly.
7. **"Hilt @Singleton for shared (DB, OkHttp); @HiltViewModel; @TestInstallIn replaces modules in tests."** Solves DI.

The 5 anti-patterns to NEVER say

1. "I'll use GlobalScope.launch."
2. "I'll just call the API in onCreate."
3. "I'll fetch on every screen open."
4. "I'll use Double for prices."
5. "I'll add tests / handle errors / think about offline later."

The single most powerful framing

When the interviewer asks anything design-y: start with **"The single source of truth is X. Reads observe a Flow from X. Writes go to X first then queue an outbox for sync. The outbox guarantees delivery across process death; WorkManager respects battery; conflicts are explicit and resolvable."** — this single template covers 80% of mobile system design questions.

Tomorrow morning

- Re-read the 90-sec opening (§2) once — say it out loud
- Re-read the 10 universal answers (§1) once
- That's it. No new material.
- Eat a real breakfast. Drink water. Walk.
- You've prepared. Trust the work.

Sleep 7+ hours. The interview will go well.